**Aim: To create a REST API using TypeScript and MongoDB and perform CRUD operations.**

## Code (server.ts)

```
import express, { Request, Response } from "express";
import mongoose from "mongoose";
import strawhatRoutes from "./routes/strawhatRoutes";

const app = express();
const PORT = 5000;

app.use(express.json());

app.get("/", (req: Request, res: Response) => {
    res.json({
        message: "One Piece - Strawhat Pirates API",
        version: "1.0.0",
        endpoints: {
            "GET /strawhats": "Get all Strawhat members",
            "POST /strawhats": "Add new Strawhat member",
            "GET /strawhats/:id": "Get member by ID",
            "GET /strawhats/name/:name": "Get member by name",
            "GET /strawhats/role/:role": "Get members by role",
            "GET /strawhats/bounty/top": "Get members sorted by bounty",
            "PUT /strawhats/:id": "Update member",
            "PATCH /strawhats/:id/bounty": "Update bounty only",
            "PATCH /strawhats/:id/alias": "Update alias only",
            "DELETE /strawhats/:id": "Remove member",
        },
    });
});

app.use("/strawhats", strawhatRoutes);

app.use((req: Request, res: Response) => {
    res.status(404).json({ error: "Route not found" });
});

const connectDB = async () => {
    try {
        const mongoURI = "mongodb://127.0.0.1:27017/onepiece";

        if (!mongoURI) {
            throw new Error(
                "MONGODB_URI is not defined in environment variables",
            );
        }

        console.log("Attempting to connect to MongoDB...");

        const options = {
            serverSelectionTimeoutMS: 5000,
            socketTimeoutMS: 45000,
            family: 4,
            retryWrites: true,
```

```
            retryReads: true,
            maxPoolSize: 10,
        };

        await mongoose.connect(mongoURI, options);
        console.log("Connected to MongoDB successfully!");
        console.log(
            `Database: ${mongoose.connection.db?.databaseName || "Unknown"}`,
        );
    } catch (error) {
        console.error("Failed to connect to MongoDB:");
        console.error(error);

        console.log("Retrying connection in 5 seconds...");
        setTimeout(connectDB, 5000);
    }
};

mongoose.connection.on("connected", () => {
    console.log("Mongoose connected to MongoDB");
});

mongoose.connection.on("error", (err) => {
    console.error("Mongoose connection error:", err);
});

mongoose.connection.on("disconnected", () => {
    console.log("Mongoose disconnected. Attempting to reconnect...");
});

const startServer = async () => {
    await connectDB();

    app.listen(PORT, () => {
        console.log(`Server running on http://localhost:${PORT}`);
        console.log(`Strawhat API: http://localhost:${PORT}/strawhats`);
    });
};

process.on("SIGINT", async () => {
    await mongoose.connection.close();
    console.log("MongoDB connection closed through app termination");
    process.exit(0);
});

startServer();

export default app;
```

## Code (/models/Strawhat.ts)

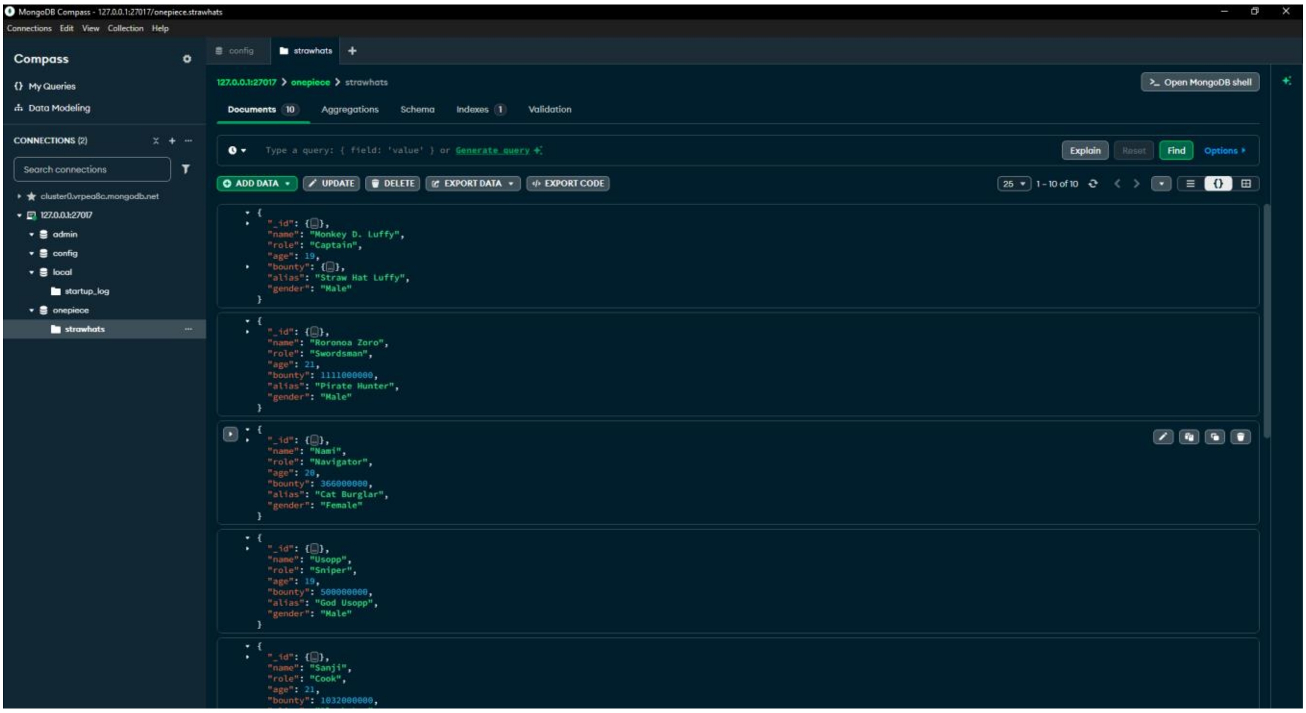
```
import mongoose from "mongoose";

const strawhatSchema = new
mongoose.Schema(
```

```
{
    name: {
        type: String,
        required: true,
```

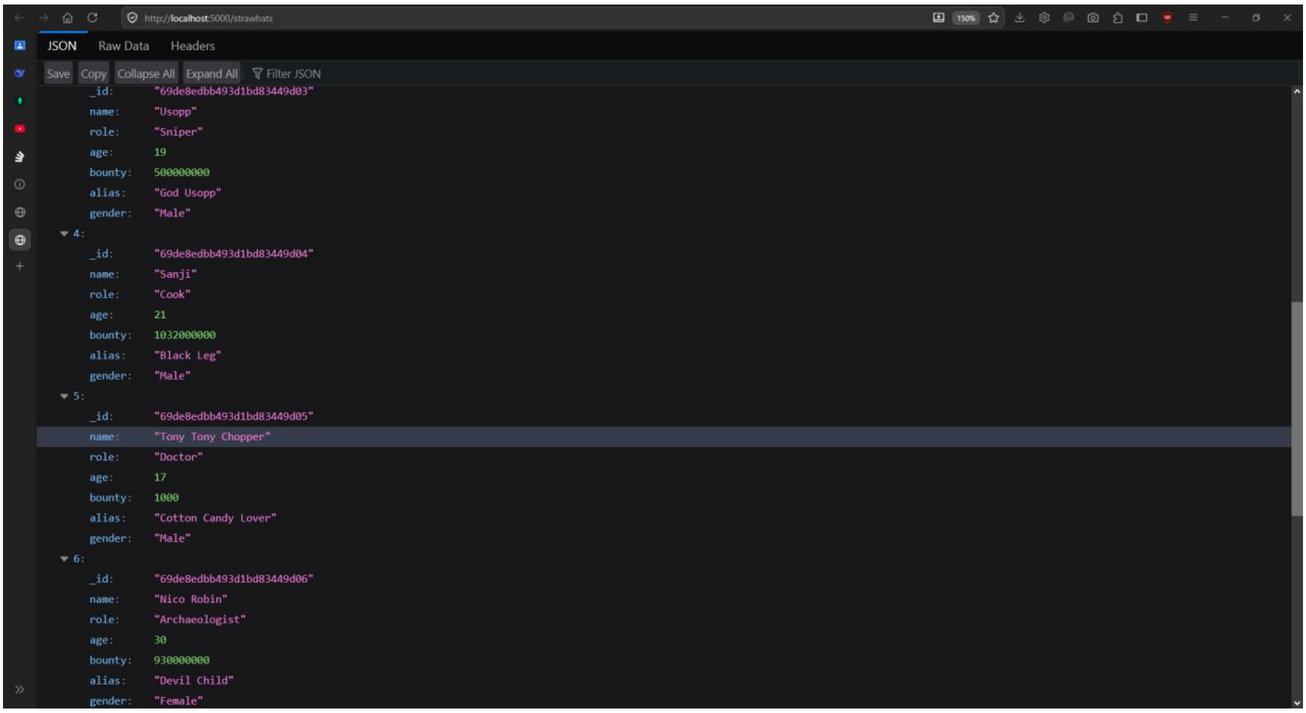
```
        trim: true,
    },
    role: {
        type: String,
        required: true,
        enum: [
            "Captain",
            "Swordsman",
            "Navigator",
            "Sniper",
            "Cook",
            "Doctor",
            "Archaeologist",
            "Shipwright",
            "Musician",
            "Helmsman",
        ],
    },
    age: {
        type: Number,
        min: 0,
        max: 200,
    },
    bounty: {
        type: Number,
        default: 0,
        min: 0,
    },
    alias: {
        type: String,
        trim: true,
    },
    gender: {
        type: String,
        enum: ["Male", "Female",
"Other"],
        required: true,
    },
  },
  {
    timestamps: true,
  },
);

export default mongoose.model("Strawhat",
strawhatSchema);
```

**Output:**

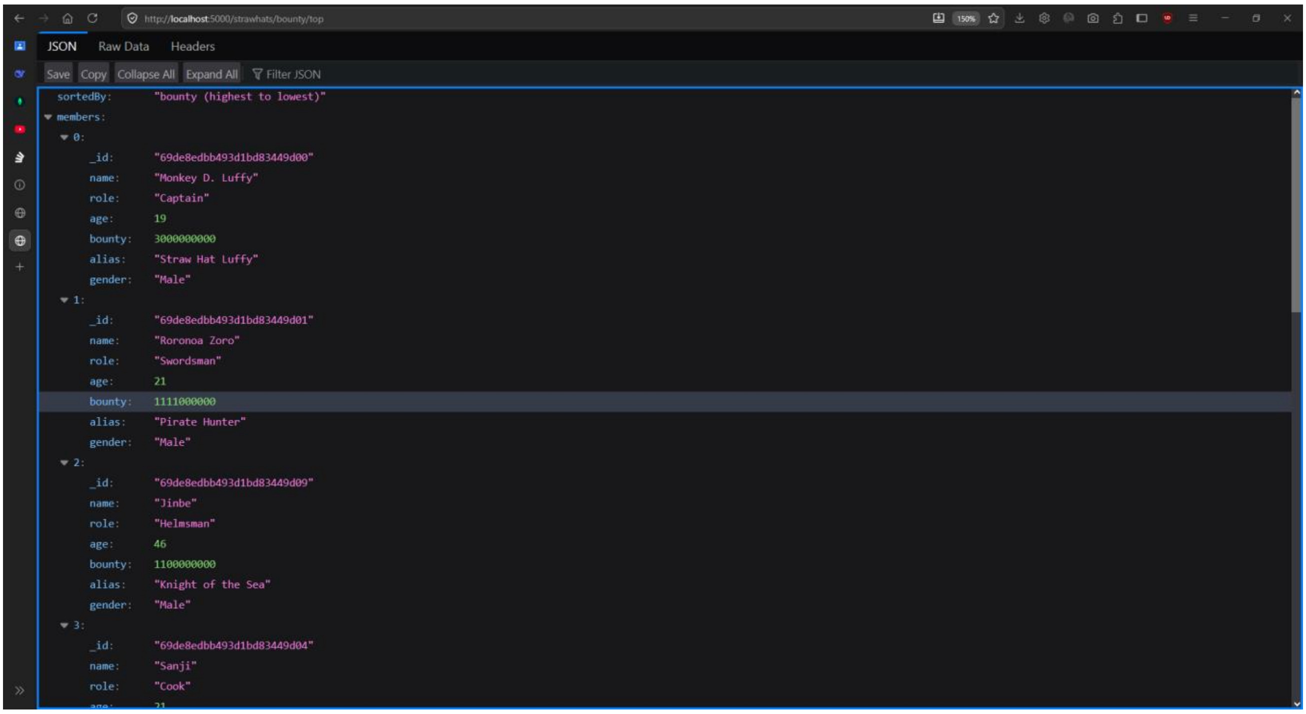
Added data to the collection 'strawhats'



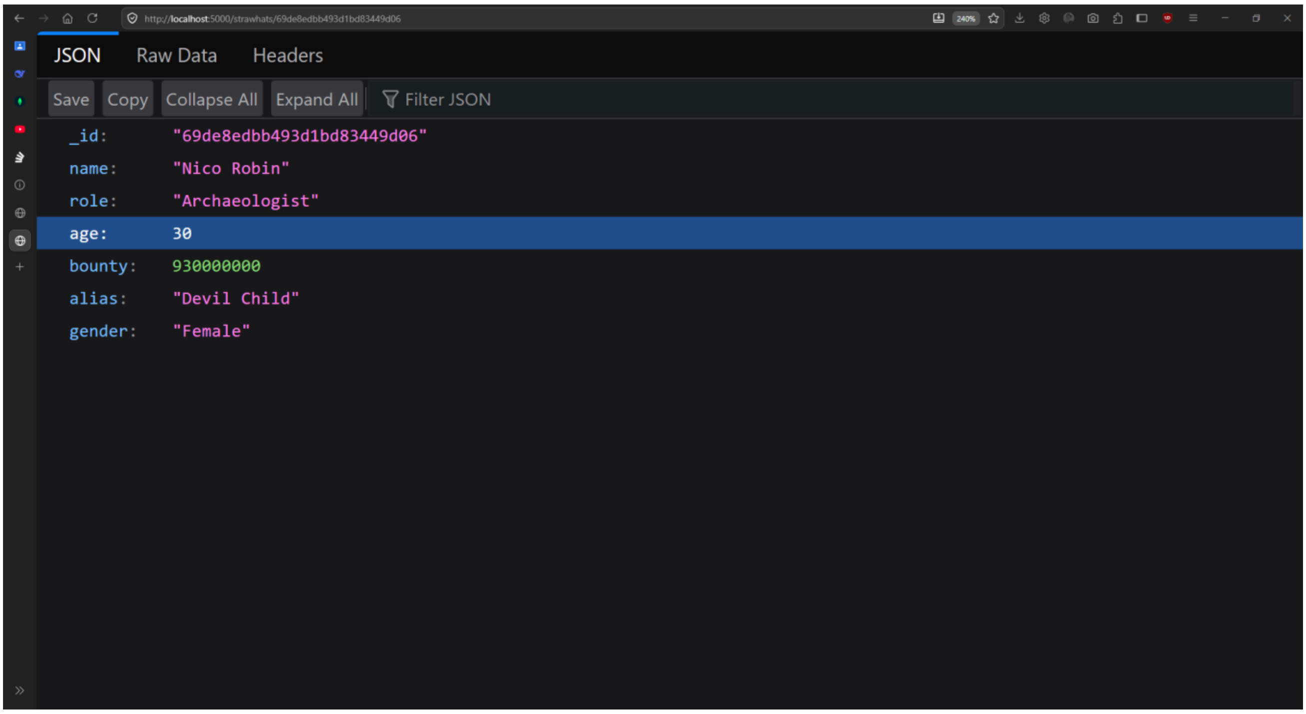
Fetched all strawhats



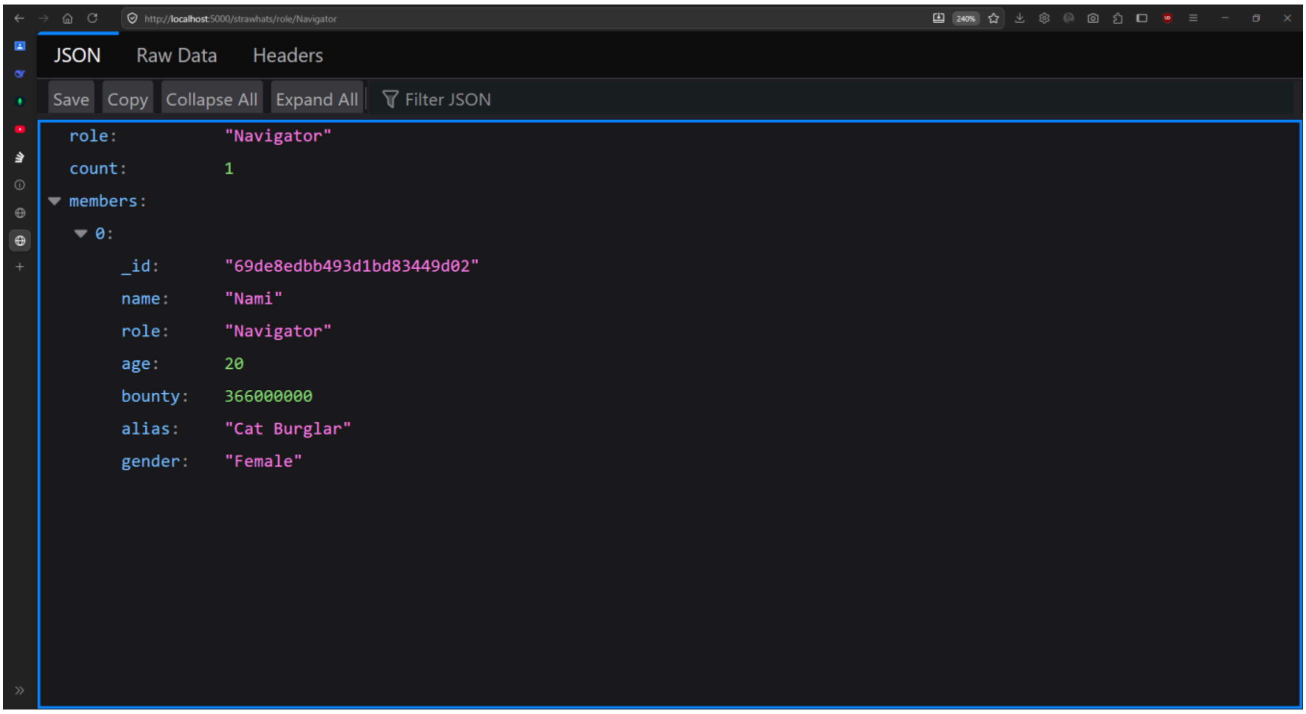

Fetched by highest bounty sort



Fetched by id




Fetched by role




## Conclusion:

The experiment demonstrates how to build a REST API using TypeScript and MongoDB and perform database operations.